Fix matching of non-type template parameters when matching template template parameters

Document #:	P3579R1 [Latest] [Status]
Date:	2025-01-15
Project:	Programming Language C++
Audience:	Core Working Group

Reply-to:	Matheus Izvekov <mizvekov@gmail.com>

Contents

§ 1 Introduction

This paper is a follow-up to [P3310R5], and proposes to fix one additional issue which came with the incorporation of [P0522R0] into the standard.

The wording change failed to prevent a narrowing conversion for non-type template parameters when matching a template *template-argument* to a *template-parameter*.

This goes against the intent of P0522, which while it preserved the type-theoretically incorrect parameter pack exception, aimed to not add new such cases when matching template template parameters.

§ 2 Changelog

§ 2.0.1 Since R0

— Added new section on partial ordering.

— Wording improvements.

§ 3 Overview

Consider the following example:

```
template<template<short> class> struct A {};

template<short> struct B;
template struct A<B>; // OK, exact match

template<int> struct C;
template struct A<C>; // #1: OK, all 'short' values are valid 'int' values

template<char> struct D;
template struct A<D>; // #2: error, not all 'short' values are valid
        'char' values
```

The intention was that [P0522R0] would allow #1, but it inadvertently also allowed #2: the wording change did not implement the paper's intent in this case, which assumed that the 'at least as specialized' check would only imply that "any template argument list that can legitimately be applied to the template template-parameter is also applicable to the argument template".

The new rules delegated this matching to partial ordering of function templates, where a rewrite produced one each for the template parameter and the template argument.

But when matching these function templates against each other in the narrowing case, the template arguments produced from the non-type template parameters are value-dependent, and narrowing conversions are not diagnosed in this case, as in general dependent entities are not diagnosed if they have valid instantiations.

§ 4 Partial ordering

This issue also manifests itself in partial ordering:

```
template<template<short> class TT1> void f(TT1<0>); // #1
template<template<int> class TT2> void f(TT2<0>);   // #2

template<int> struct B;
template void f<B>(B<0>); // selects #2
```

Before P0522, this selected #2.

After P0522, B matches both overloads, but they match each other during partial ordering, so this is now ambiguous.

The change proposed in this paper will make it so `#1` doesn't match `#2`, making `#2` more specialized, restoring the original semantics.

§ 5 Wording

Change 13.4.4 [temp.arg.template]/4

— Each function template has a single function parameter whose type is a specialization of X with template arguments corresponding to the template parameters from the respective function template where, for each template parameter PP in the template-head of the function template, a corresponding template argument AA is formed. If PP declares a template parameter pack, then AA is the pack expansion PP… ([temp.variadic]); otherwise, AA is the id-expression PP.

— For each non-type template parameter PA from the *template-head* of A, if a narrowing conversion ([dcl.init.list]) is required for initializing a variable of the same type as the corresponding parameter of the *template-head* of P from a variable of the same type as PA, the rewrite is ill-formed.

If the rewrite ~~produces an invalid type~~is ill-formed, then P is not at least as specialized as A.

[Example:

```
template<template<short> class P> struct S {};
template<int> struct A;
template struct S<A>; // OK, not narrowing
```

Rewrite of the matching of the above template template parameter, in terms of function template partial ordering:

```
template<int> struct X {};          // Invented class X with template-head
        of A
template<short PP> void f(X<PP>); // #P
template<int PP> void f(X<PP>);   // #A
template  void  f<0>(X<0>);               //  Invoke  partial  ordering  for
        exposition only. OK: selects #P
```

– end example]

[Example:

```
template<template<short> class P> struct S {};
template<char> struct B;
template struct S<B>; // error: narrowing
```

Rewrite of the matching of the above template template parameter, in terms of function template partial ordering:

```
template<char> struct X {};        // Invented class X with template-head
        of B
template<short PP> void f(X<PP>); // #P
template<char PP> void f(X<PP>);  // #A
template  void  f<0>(X<0>);               //  Invoke  partial  ordering  for
        exposition only. Bad: narrowing conversion on #P
```

As ordinary code, `#P` is valid, and partial ordering would select `#P`, but in the context of the rewrite, the narrowing of PP to char is ill-formed.

– end example]

[Example:

```
template<template<short> class> void f(); // #1
template<template<int> class> void f();   // #2

template<int> struct B;
template void f<B>(); // selects #2
```

– end example]

§ 6 References

[P0522R0] James Touton, Hubert Tong. 2016-11-11. DR: Matching of template template-arguments excludes compatible templates.

https://wg21.link/p0522r0

[P3310R5] Matheus Izvekov. 2024-11-18. Solving partial ordering issues introduced by relaxed template template parameter matching.

https://wg21.link/p3310r5